Handling Missing Data: Imputing Numerical Columns with SimpleImputer

Contents

1. Overview . . . 2
2. Mean and median imputation . . . 3
   - 2.1 Mean or median . . . 3
   - 2.2 Advantages . . . 5
   - 2.3 Disadvantages . . . 5
   - 2.4 When to use it . . . 5
3. Mean and median imputation on real data . . . 6
   - 3.1 The data and the split . . . 6
   - 3.2 Variance shrinks . . . 6
   - 3.3 The distribution changes shape . . . 7
   - 3.4 Covariance and correlation change . . . 8
   - 3.5 New outliers appear . . . 10
   - 3.6 The verdict . . . 10
4. Imputation with scikit-learn's SimpleImputer . . . 11
   - 4.1 The main parameters . . . 11
   - 4.2 A different strategy per column . . . 12
5. Arbitrary value imputation . . . 12
   - 5.1 Advantages, disadvantages and when to use it . . . 13
   - 5.2 On real data . . . 13
6. End of distribution imputation . . . 14
   - 6.1 Normal columns: mean plus three standard deviations . . . 15
   - 6.2 Skewed columns: the IQR rule . . . 15
   - 6.3 On real data . . . 16
   - 6.4 Advantages, disadvantages and when to use it . . . 17
7. Summary . . . 18
8. Sources . . . 18
9. Key terms . . . 19

Where this fits: Pipeline map step 4 (Clean). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	**4 Clean**	5 Features	6 Reduce
7 Split	8 Model	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Train-test split; Missing values.
- **Leads to:** Iterative imputation (MICE); Grid and random search.
- **Compare with:** Complete case analysis; Random sample imputation; KNN imputer.

1. Overview

Key point: We fill each gap in a numerical column with one number taken from the same column: its mean or median, an arbitrary value, or a value at the end of its distribution.

Where each technique is covered mapped the ways of handling missing data. This Note covers univariate imputation of numerical features. A **feature** (G-772) is an input variable (one column of the data table), the **target** (G-1949) is the output we predict, and an **observation** (G-1374) is one record (one row). In univariate imputation, each gap in a feature is filled using only the other values of that same feature.

Figure 1 shows the three techniques on a small `Age` column with two gaps (the gaps are the missing values, marked `NaN`, "not a number"). Read each column after the first as the same `Age` column after one technique (the second column covers both mean and **median imputation** (G-1208)):

- **Mean or median imputation** fills each gap with a typical value from the centre of the column.
- **Arbitrary value imputation** (G-208) fills each gap with a value that never occurs, such as 99 or -1.
- **End of distribution imputation** (G-686) fills each gap with a value at the edge of the column's distribution.

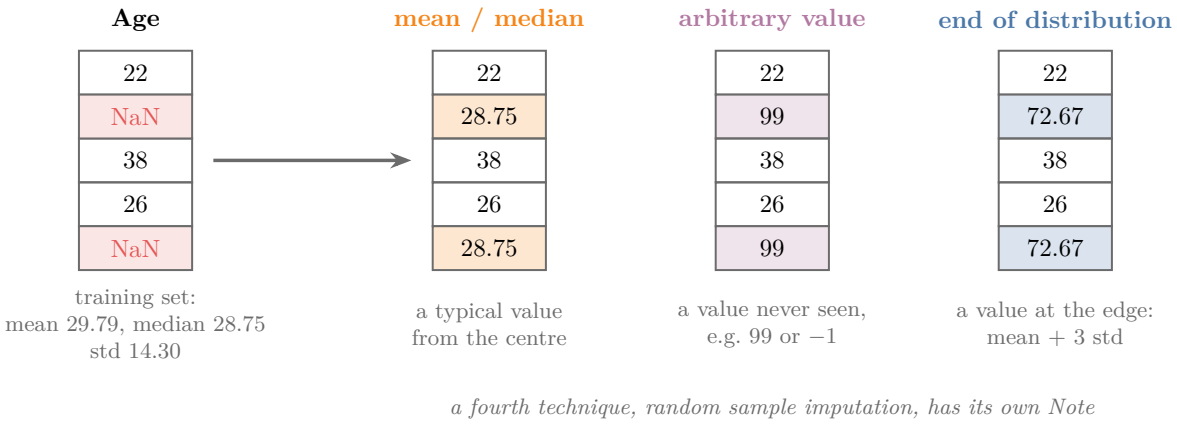


Figure 1: Three ways to fill the gaps of a numerical column; the fill values come from the training set

A fourth univariate technique, **random sample imputation** (G-1616), has its own section, together with a way to pick the best technique automatically. scikit-learn's `SimpleImputer` does all three techniques of this Note.

After every imputation we check what it did to the column. We compare four things before and after: the **variance** (G-2074), the distribution, the covariance and correlation with other columns, and the outliers.

2. Mean and median imputation

> **Key point:** Mean or median imputation replaces every missing value with the mean or the median of the known values of the column.

Mean imputation (G-1197) replaces each missing value with the mean of the column. **Median imputation** (G-1208) replaces it with the median. Both are computed from the values that are present.

2.1 Mean or median

> **Key point:** Use the mean when the column is roughly normal (symmetric), and the median when it is skewed.

For a normal distribution, the mean, median and mode are all equal, so either choice gives the same value. For a skewed distribution, the long tail pulls the mean towards it, away from the bulk of the data. The median stays in the middle of the data, so it is the better choice there.

Figure 2 shows the pull on nine real fares from the training data of Section 3 (the smallest fare, the largest, and seven spread evenly between them). Their mean, 75.5, is higher than eight of the nine fares, because the one fare of 512 drags it up. Their median, 14.46, sits

inside the crowd. Now raise the largest fare step by step to 5,000: the mean runs after it, to 574.1, while the median does not move at all. The median only looks at which value is in the middle, not at how far away the largest one is.

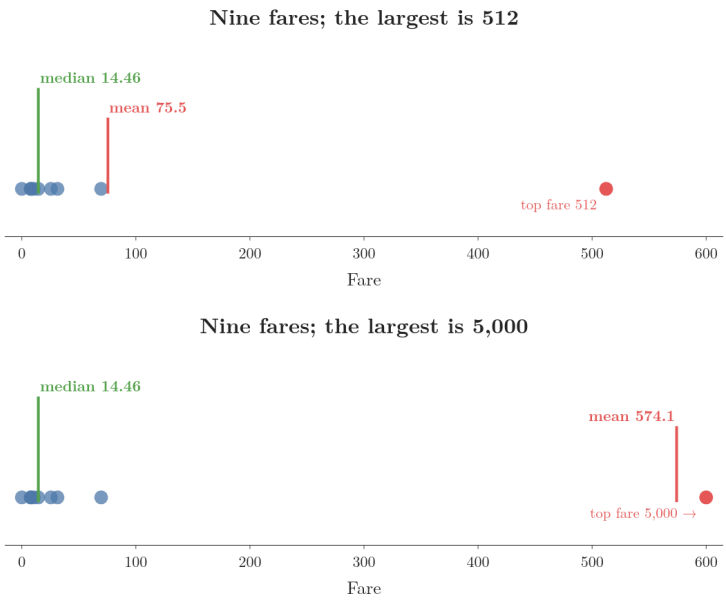


Figure 2: Nine training fares on a number line. As the largest fare is raised from 512 to 5,000, the mean moves from 75.5 to 574.1; the median stays at 14.46. Idea after Khan Academy, "Mean and standard deviation versus median and IQR".

Figure 3 then shows both cases on all of the Titanic training data of Section 3. `Age` is close to symmetric: its mean (29.79) and median (28.75) almost coincide. `Fare` is strongly right-skewed: a few very expensive tickets pull the mean up to 32.62, more than twice the median of 14.46.

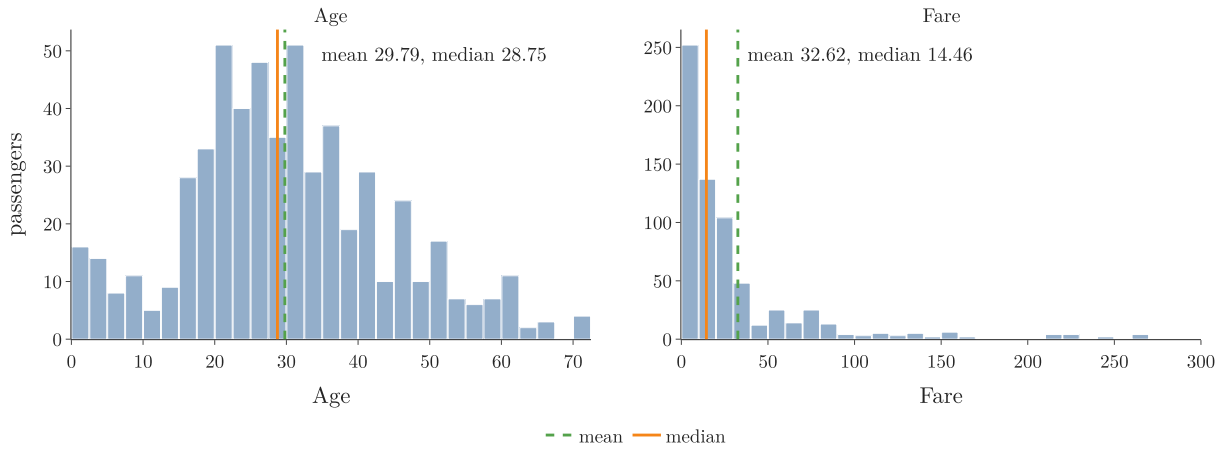


Figure 3: Mean and median of Age (nearly symmetric) and Fare (right-skewed, shown up to 300; the largest fare is 512)

2.2 Advantages

> **Key point:** Mean or **median imputation** (G-1208) is simple to compute and simple to repeat on new data in production.

1. **Simple.** We compute one number per column and put it into every gap.
2. **Easy to use in production.** The deployed model receives new **observations** (G-1374) that may have gaps. We store the mean or median from training and fill the new gaps with it.

Mean or median imputation gives reasonable results when few values are missing, under about 5% of the **feature** (G-772). With more missing values the method becomes less reliable.

2.3 Disadvantages

> **Key point:** Mean or **median imputation** (G-1208) piles values at the centre, which changes the distribution's shape, creates outliers, and weakens the feature's relationships with other features.

1. **The shape of the distribution changes.** Every gap gets the same value, so a tall spike appears at the centre.
2. **Outliers appear.** With many values at the centre, the middle half of the data becomes narrower. Values that were normal before now fall outside the whiskers of a box plot and count as outliers.
3. **The covariance and correlation with other features change.** The filled values ignore the other **features** (G-772), so the feature's relationship with them weakens.

All three are bad for a model. Section 3 shows each of them on real data.

2.4 When to use it

> **Key point:** Use mean or **median imputation** (G-1208) when the data is missing completely at random and each column is missing under about 5% of its values.

The two conditions are the same as for **complete case analysis** (G-425):

1. **The data is MCAR** (G-1192, missing completely at random: whether a value is missing has nothing to do with any data; see **MCAR** (G-1192)).
2. **Under about 5% of the column is missing.**

MCAR matters because the fill value is computed from the known values only. When the gaps are not random, the known values are a lopsided sample: in the MNAR example, hiding short experience raised the mean of the known values from 9.9 to 11.9 years. Every gap would then get a value that is too high. The 5% limit matters because the damage of Section 3 grows with the number of filled values.

Mean and median imputation is used widely because it is so simple. Better techniques exist, so in practice we try several and keep the one that works best.

3. Mean and median imputation on real data

> **Key point:** On the Titanic data, mean or **median imputation** (G-1208) leaves `Fare` (5% missing) almost unchanged but distorts `Age` (21% missing).

3.1 The data and the split

> **Key point:** We split into training and test sets first, and compute every fill value from the training set only.

The data has 891 Titanic passengers, one **observation** (G-1374) each, and four numerical columns: three features, `Age`, `Fare` and `Family`, and the **target** (G-1949) `Survived`. `Family` is the number of siblings, spouses, parents and children aboard.

`Age` has 177 missing values (19.9%). `Fare` originally had none; 45 values (5.1%) were deleted at random so that it has some. `Family` and `Survived` are complete.

We split the data 80/20: 712 training rows and 179 test rows. In the training set, 20.8% of `Age` and 5.1% of `Fare` is missing.

> **Python:** Split first, then compute the fill values on the training set.
> ```
> X_train, X_test, y_train, y_test = train_test_split(
>     X, y, test_size=0.2, random_state=2)
> X_train = X_train.copy()
> median_age = X_train["Age"].median()    # 28.75
> mean_age = X_train["Age"].mean()        # 29.79
> X_train["Age_median"] = X_train["Age"].fillna(median_age)
> X_train["Age_mean"] = X_train["Age"].fillna(mean_age)
> ```
> **`fillna(value)`** (G-82) returns the column with every `NaN` replaced by `value`. We keep the original column next to the filled ones to compare them. `copy()` makes `X_train` an independent table, so adding columns gives no `SettingWithCopyWarning`.

The same is done for `Fare`, with median 14.46 and mean 32.62. The test set is filled later with these same training values.

> **Extra:** Computing the mean on the whole table before the split lets the test rows influence the fill value. The test set would then no longer be truly unseen data. Letting test rows shape the fill value is called **data leakage** (G-535), and the rule is always: fit on the training set, apply to both sets (scikit-learn docs, Data leakage).

3.2 Variance shrinks

> **Key point:** Mean or **median imputation** (G-1208) always lowers the variance, and the more values are missing, the more it drops.

The **variance** (G-2074) measures the average squared distance from the mean. Every filled value sits at or near the mean, so it adds almost nothing to that distance, and the variance

falls. An everyday picture: fill every blank in a class's height list with the average height, and the class suddenly looks more alike than it really is. Figure 4 shows it on the training ages: the 148 filled values form one tall bar at the mean, and the band of one standard deviation narrows.

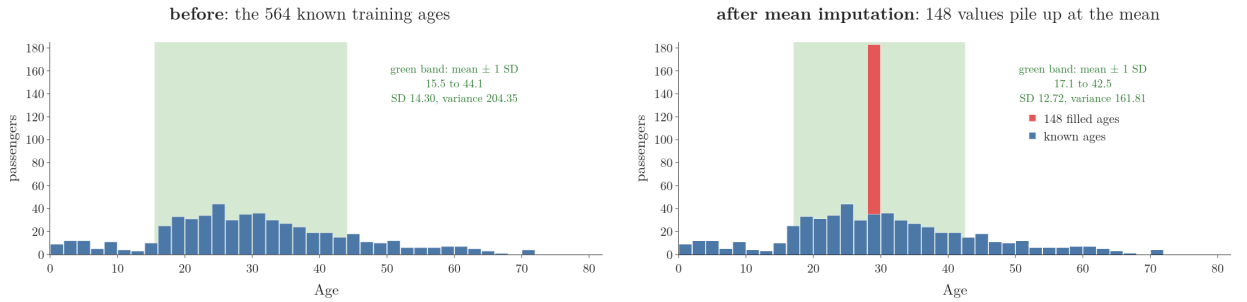


Figure 4: The 712 training ages before and after mean imputation. The 148 filled ages (red) stack at the mean; the band of one standard deviation narrows from 15.5–44.1 years to 17.1–42.5 years.

Column	Original	Median imputed	Mean imputed
`Age` (20.8% missing)	204.35	161.99	161.81
`Fare` (5.1% missing)	2,448.20	2,340.09	2,324.24

`Age` lost about a fifth of its variance; `Fare` lost only about 5%. A small change is acceptable; a large one is a warning sign.

Extra: For **mean imputation** (G-1197) we can predict the new variance exactly.

1. **In words:** the filled values add no squared distance, so the same sum of squared distances is now divided by the larger count.
2. **Formula:** with k known values out of n rows,

$$s^2_{\text{after}} = s^2_{\text{before}} \times \frac{k-1}{n-1}$$

3. **Example:** `Age` has $k = 564$ known values out of $n = 712$:

$$204.35 \times \frac{563}{711} = 161.81,$$

exactly the value in the table.

3.3 The distribution changes shape

Key point: For `Age`, a tall peak appears at the fill value; for `Fare`, the curves before and after lie on top of each other.

Figure 5 draws the density curve (KDE, a smooth curve over a histogram; see the density plot) of each column before and after imputation. Blue is the original column with its gaps skipped; orange is median imputed; green dashed is mean imputed.

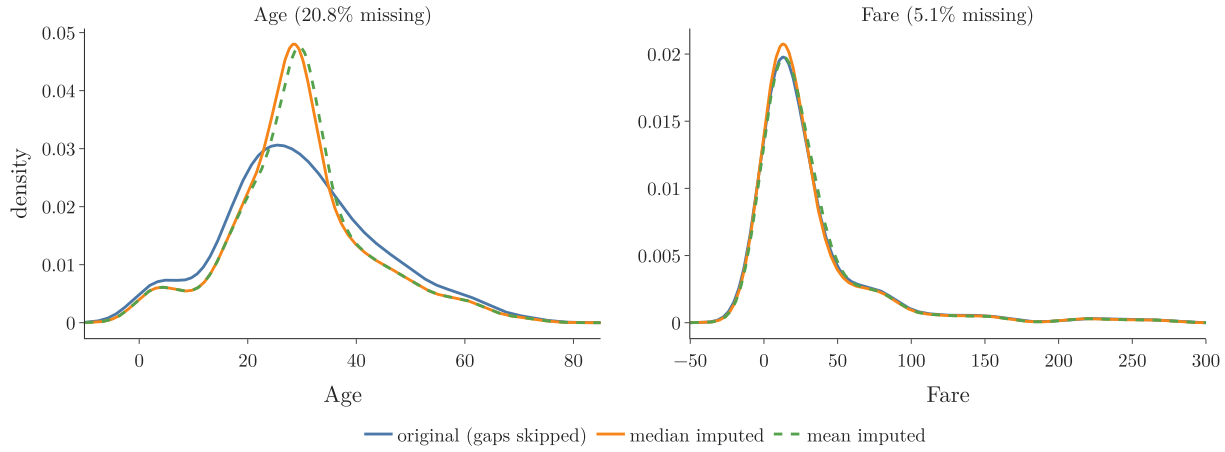


Figure 5: Density of Age and Fare before and after mean and median imputation: Age gains a tall peak at the centre, Fare barely changes

For `Age`, the peak grows from about 0.03 to about 0.05, and the curve becomes narrower and taller. The 148 filled values all sit at 28.75 or 29.79. For `Fare`, the three curves almost overlap, because only 36 values were filled.

3.4 Covariance and correlation change

> **Key point:** A filled value ignores the other **features** (G-772), so imputing `Age` weakens its link with them: its correlation with `Family` falls by about a fifth.

Covariance (G-496) measures how two features move together: positive if they rise together, negative if one rises as the other falls. Its size depends on the units, so it has no fixed limits. **Correlation** (G-490) is the covariance rescaled to lie between -1 and 1. A worked case with three passengers: ages 20, 30, 40 (mean 30) and family sizes 3, 1, 2 (mean 2).

Passenger	Age minus mean	Family minus mean	Product
1	-10	1	-10
2	0	-1	0
3	10	0	0

$$\text{covariance} = \frac{-10 + 0 + 0}{3 - 1} = -5$$

The age has a spread (standard deviation) of 10 and the family size a spread of 1, so

$$\text{correlation} = \frac{-5}{10 \times 1} = -0.5$$

The negative sign says that older passengers here have smaller families.

Covariance with	`Fare`	`Family`
`Age` original	70.72	−6.50
`Age` median imputed	57.96	−5.11
`Age` mean imputed	55.60	−5.15

Correlation with	`Fare`	`Family`
`Age` original	0.093	−0.299
`Age` median imputed	0.092	−0.243
`Age` mean imputed	0.088	−0.245

The relationship with `Family` weakened by about a fifth. Figure 6 shows why. Every filled age sits at the same value, 29.8, whatever the passenger's family size, so the 148 filled points form a flat line with no trend. Mixed in with the real points, that flat line pulls the correlation towards 0.

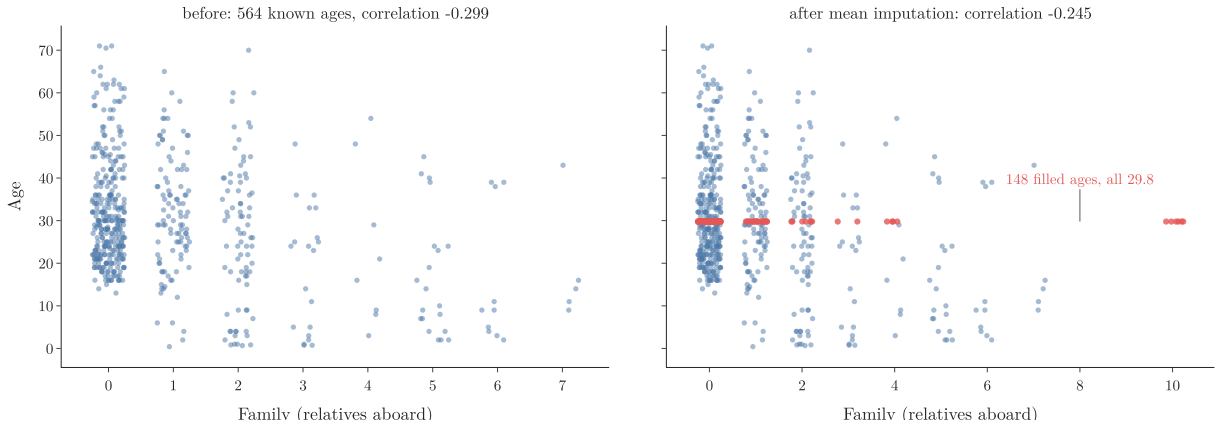


Figure 6: Age against Family in the 712 training rows (jittered sideways). Left: the 564 known ages. Right: after mean imputation, the 148 filled ages (red) all sit at 29.8 and carry no trend, so the correlation weakens from −0.299 to −0.245.

For `Fare`, the covariance with `Family` only moved from 17.26 to 16.48 (median) and 16.39 (mean).

> **Python:** The whole covariance or correlation table in one call.
> `X_train.cov()`

`X_train.corr()`
Both skip missing values pair by pair (pandas docs, DataFrame.cov). The "original" numbers therefore use only the rows where both columns are known.

3.5 New outliers appear

Key point: Imputing `Age` narrows its box, and the number of outliers jumps from 7 to 69.

Figure 7 shows box plots before and after. A box plot draws the middle half of the data as a box, from the first quartile (Q1) to the third (Q3). Points more than 1.5 box-lengths beyond the box are outliers (see whiskers and outliers).

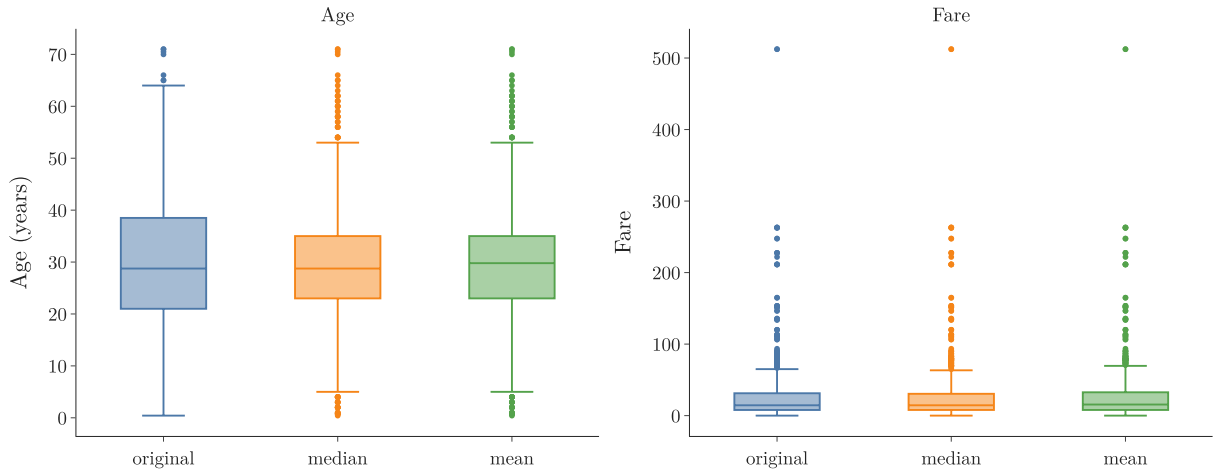


Figure 7: Box plots before and after imputation: the Age box narrows and many new outliers appear on both sides; Fare is unchanged

For `Age`, the box was 21 to 38.25. After imputation it is 23 to 35, because so many values now sit at the centre. Passengers aged 54 or 4, normal before, are now outliers on both sides.

For `Fare`, the box hardly moves. The count of outliers goes from 93 to 94 (median) and 84 (mean).

3.6 The verdict

Key point: Mean imputation (G-1197) is fine for `Fare`; for `Age`, we should look for a better technique.

Check	`Age`	`Fare`
Variance	falls by a fifth	falls by about 5%

Check	`Age`	`Fare`
Distribution	tall new peak	unchanged
Covariance, correlation	weakened	almost unchanged
Outliers	7 become 69	unchanged

`Fare` shows no warning signs, so mean (or median) imputation is a reasonable choice. `Age` fails every check, which matches Section 2.4: about 21% of it is missing, far above 5%.

4. Imputation with scikit-learn's SimpleImputer

> **Key point:** `SimpleImputer` learns the fill value with `fit` on the training set and fills gaps with `transform`; inside a `ColumnTransformer`, each column can get its own strategy.

pandas `fillna` is easy, but scikit-learn's `SimpleImputer` is the better tool for real projects. `SimpleImputer` can go into a pipeline (a chain of steps run in order; see pipelines), can be tuned with grid search, and its learned values are saved with the model for production.

4.1 The main parameters

> **Key point:** `strategy` picks the fill value; `fill_value` sets it when the strategy is `"constant"`.

Parameter	Meaning	Default
`missing_values`	what counts as missing	`np.nan`
`strategy`	`"mean"`, `"median"`, `"most_frequent"` or `"constant"`	`"mean"`
`fill_value`	the value used with `"constant"`	0 for numbers
`add_indicator`	also add a 0/1 column marking each gap	`False`
`keep_empty_features`	keep a column that is missing everywhere	`False`

`"most_frequent"` fills with the mode, used mainly for categorical columns (missing categorical data). `"constant"` is used for arbitrary value and **end of distribution imputation** (G-686) (Sections 5 and 6).

> **Extra:** `add_indicator=True` adds the **missing indicator** (G-1233; a 0/1 column that marks each gap), covered in the missing indicator.

4.2 A different strategy per column

> **Key point:** A `ColumnTransformer` applies the median to `Age`, the mean to `Fare`, and passes `Family` through unchanged.

> **Python:** Two imputers, one per column.
> ```
> trf = ColumnTransformer([
>     ("imputer1", SimpleImputer(strategy="median"), ["Age"]),
>     ("imputer2", SimpleImputer(strategy="mean"), ["Fare"]),
> ], remainder="passthrough", verbose_feature_names_out=False)
> trf.set_output(transform="pandas")
> trf.fit(X_train)
> X_train = trf.transform(X_train)
> X_test = trf.transform(X_test)
> ```
> The other columns pass through unchanged, because of `remainder="passthrough"`. **`set_output(transform="pandas")`** (G-1780) makes `transform` return a DataFrame with column names instead of a bare numpy array.

Figure 8 runs the two steps on six test rows with gaps: `fit` learns the training median of `Age` and the training mean of `Fare`, and `transform` writes those values into every gap, test rows included.

Six test rows with gaps (red)

test row	Age	Fare	Family
692	NaN	56.4958	0
643	NaN	56.4958	0
65	NaN	15.2458	2
702	18	NaN	1
128	NaN	22.3583	2
355	28	NaN	0

transform: every gap gets the training value (orange)

test row	Age	Fare	Family
692	28.75	56.4958	0
643	28.75	56.4958	0
65	28.75	15.2458	2
702	18	32.6176	1
128	28.75	22.3583	2
355	28	32.6176	0

Figure 8: SimpleImputer on six test rows with gaps (red). fit learns median Age 28.75 and mean Fare 32.62 from the training set; transform fills every gap with them (orange).

After `fit`, each imputer stores its learned value in **`statistics_`** (G-145):

```
trf.named_transformers_["imputer1"].statistics_   # [28.75]
trf.named_transformers_["imputer2"].statistics_   # [32.6176]
```

These two numbers are the training median of `Age` and the training mean of `Fare`. `transform` puts the same 28.75 and 32.62 into the gaps of the training set *and* the test set.

5. Arbitrary value imputation

> **Key point:** Fill every gap with one value that never occurs in the column, so the model can tell missing rows apart from the rest.

Arbitrary value imputation replaces every missing value with one fixed value chosen by us, such as 99, 999 or −1. The value must not occur in the real data. For `Age`, no passenger is 99 or −1 years old.

The aim is the opposite of **mean imputation** (G-1197). Instead of hiding the gaps, we mark them, so the model can learn whether "missing" itself carries information.

The technique is used mostly for categorical **features** (G-772), where gaps become a new category such as "Missing" (missing categorical data). **Arbitrary value imputation** (G-208) works the same way for numbers.

5.1 Advantages, disadvantages and when to use it

> **Key point:** Arbitrary value imputation is easy, but it distorts the distribution, **variance** (G-2074) and covariance; use it when the data is not missing at random.

- **Advantage:** very easy to apply.
- **Disadvantages:** the same three as **mean imputation** (G-1197), usually worse. A spike appears at the arbitrary value, so the distribution (its density curve, or PDF) changes; the variance changes, often grows a lot; the covariance and correlation with other features change.
- **When to use it:** when the data is *not* missing completely at random. Then the fact that a value is missing may itself tell the model something.

The distortion is accepted on purpose: it is what makes the missing rows stand out. In practice the technique is not used much, because better ones exist.

5.2 On real data

> **Key point:** Filling `Age` with 99 multiplies its **variance** (G-2074) by 4.7; filling `Fare` with 999 multiplies it by 19.

We fill `Age` with 99 and with −1, and `Fare` with 999 and with −1, each into a new column.

Column	Original	Large value (99 / 999)	−1
`Age` variance	204.35	951.73	318.09
`Fare` variance	2,448.20	47,219.20	2,378.57

Figure 9 shows the distributions. For `Age`, each fill value creates a second hump: at 99 (red) or at −1 (purple dashed), and the main peak drops. For `Fare`, 999 flattens the curve; −1 sits next to the many cheap tickets, so its curve almost matches the original.

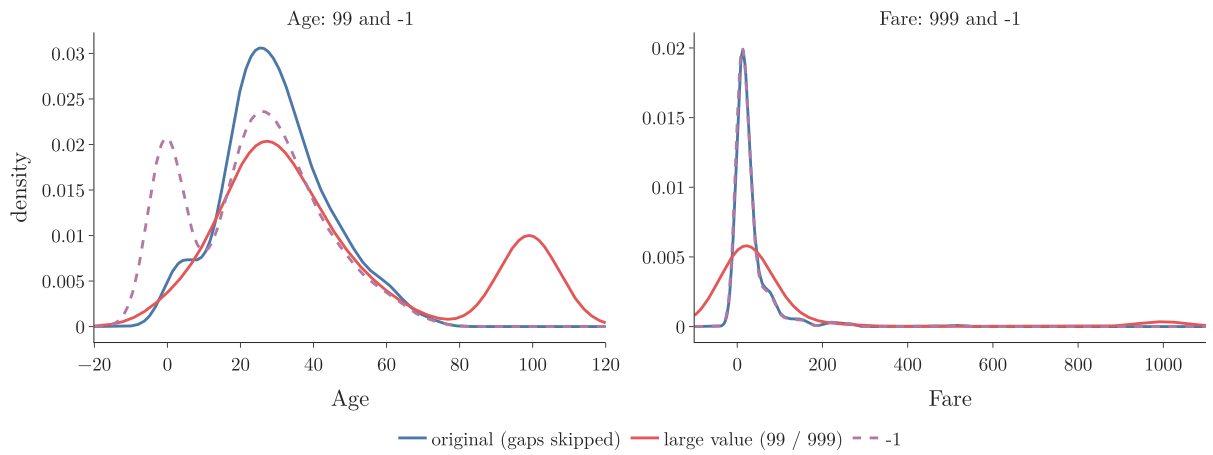


Figure 9: Density of Age and Fare after arbitrary value imputation: a new hump appears at 99 or at -1

The relationships change too. The **covariance** (G-496) of `Age` with `Fare` goes from 70.72 to −101.67 with 99, and to 125.56 with −1. The **correlation** (G-490) of `Fare` with `Family` falls from 0.208 to 0.032 with 999.

Python: Arbitrary value imputation (G-208) with `SimpleImputer`.

```
trf = ColumnTransformer([
    ("imputer1", SimpleImputer(strategy="constant",
                               fill_value=99), ["Age"]),
    ("imputer2", SimpleImputer(strategy="constant",
                               fill_value=999), ["Fare"]),
], remainder="passthrough")
```

With `strategy="constant"`, `statistics_` simply holds the chosen values, 99 and 999.

6. End of distribution imputation

Key point: Instead of guessing an arbitrary value, take one from the far end of the column's own distribution.

Choosing a good arbitrary value is hard: it must be outside the data, but how far? **End of distribution imputation** answers this by filling the gaps with a value at the end of the column's distribution. The technique is an extension of **arbitrary value imputation** (G-208), and the two share the same aim: mark the missing rows for the model.

How we find the end depends on the shape of the column.

6.1 Normal columns: mean plus three standard deviations

> **Key point:** If the column is roughly normal, use the mean plus (or minus) three standard deviations.

In a normal distribution, about 68% of the values lie within one standard deviation σ of the mean, 95% within two, and 99.7% within three. With a mean of 30 and a standard deviation of 10, that means 68% of the values lie between 20 and 40, 95% between 10 and 50, and 99.7% between 0 and 60. A value beyond three standard deviations is therefore at the very end of the distribution.

The fill value, step by step:

1. **In words:** start at the mean and move three standard deviations to the right (or the left).
2. **Formula:**
$$\text{right end} = \mu + 3\sigma$$
$$\text{left end} = \mu - 3\sigma$$
Here μ is the column's mean and σ its standard deviation.
3. **Example:** the training `Age` has mean 29.79 and standard deviation 14.30. The two ends are:
$$\text{right end} = 29.79 + 3 \times 14.30$$
$$\text{right end} = 72.67$$
$$\text{left end} = 29.79 - 3 \times 14.30$$
$$\text{left end} = -13.10$$
The left end is an impossible age, so we use the right end, 72.67.

6.2 Skewed columns: the IQR rule

> **Key point:** If the column is skewed, use Q3 plus 1.5 times the IQR (or Q1 minus 1.5 times the IQR).

The fill value sits on a box-plot fence, 1.5 IQR beyond the box, as in whiskers and outliers. Values past a fence count as outliers, so a fill there stands out.

The fill value, step by step:

1. **In words:** start at the third quartile and move 1.5 box-lengths to the right (or start at Q1 and move left).
2. **Formula:**
$$\text{right end} = Q_3 + 1.5 \times \text{IQR}$$
$$\text{left end} = Q_1 - 1.5 \times \text{IQR}$$

3. **Example:** the training `Fare` has $Q_1 = 7.90$ and $Q_3 = 31.28$. The ends are:

$$\text{IQR} = 31.28 - 7.90 = 23.38$$

$$1.5 \times 23.38 = 35.07$$

$$\text{right end} = 31.28 + 35.07$$

$$\text{right end} = 66.34$$

$$\text{left end} = 7.90 - 35.07$$

$$\text{left end} = -27.17$$

The left end is an impossible fare, so we use the right end, 66.34.

6.3 On real data

> **Key point:** The end values create a hump at the edge of the distribution and change the **variance** (G-2074), just as arbitrary values do.

Figure 10 fills `Age` with 72.67 and `Fare` with 66.34. For `Age`, 148 values land at the far right, creating a second hump there, and the variance rises from 204.35 to 465.07. For `Fare`, the 36 filled values make only a small bump, and the variance moves from 2,448.20 to 2,378.92.

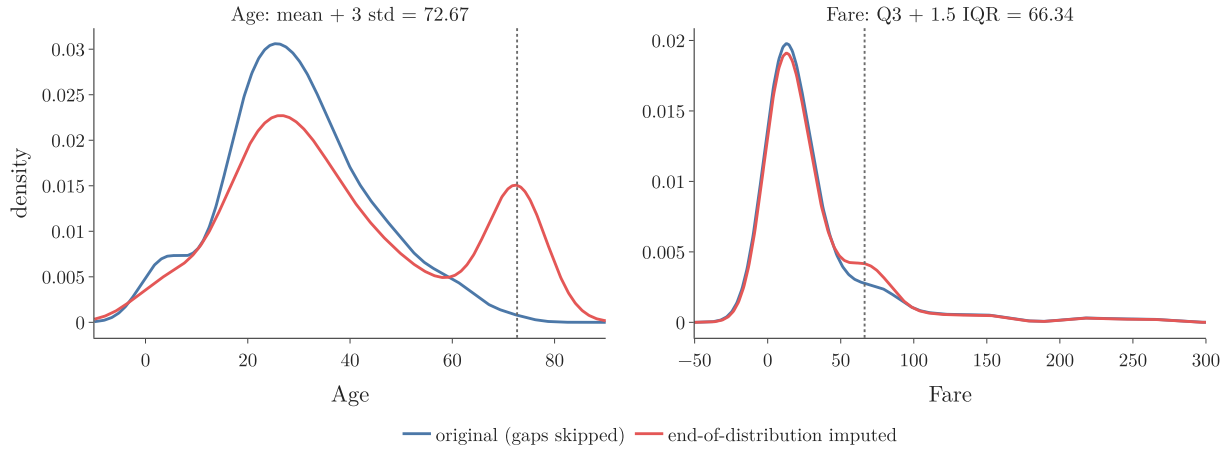


Figure 10: Density before and after end of distribution imputation; the dotted line is the fill value

> **Python:** No new tool is needed: compute the end value on the training set and pass it as `fill_value`.

```
age_end = X_train["Age"].mean() + 3 * X_train["Age"].std()
SimpleImputer(strategy="constant", fill_value=age_end)
```

6.4 Advantages, disadvantages and when to use it

> **Key point:** The same as arbitrary value imputation: easy, but it changes the distribution, **variance** (G-2074) and covariance; use it when the data is not missing at random.

- **Advantage:** easy to apply, and the value comes from the data instead of a guess.
- **Disadvantages:** the distribution changes, the **covariance** (G-496) changes, and the variance can grow.
- **When to use it:** when the data is not missing completely at random, to show the model that a value was missing.

7. Summary

Technique	Fills each gap with	`strategy`	Use when
Mean imputation	mean of the column	`"mean"`	MCAR, under 5% missing, roughly normal
Median imputation	median of the column	`"median"`	MCAR, under 5% missing, skewed
Arbitrary value	a value that never occurs (99, -1)	`"constant"`, fill 99	not MCAR: missing itself is informative
End of distribution	$\mu + 3\sigma$, or $Q_3 + 1.5\,\mathrm{IQR}$	`"constant"`, fill the end value	not MCAR, and no obvious arbitrary value

- Univariate imputation fills a column's gaps using only that column, so it is one stored number per column and easy to repeat on new data in production.
- Mean and **median imputation** (G-1208) hide the gaps at the centre; arbitrary value and end of distribution imputation mark them at the edges, so the model can learn whether "missing" itself carries information. The median suits a skewed column, because one huge value drags the mean but not the median.
- Every technique here changes the column. After imputing, compare the **variance** (G-2074), the distribution (KDE), the covariance and correlation, and the outliers (box plot), before and after, because a large change in any of them is bad for a model.
- On the Titanic data, **mean imputation** (G-1197) was fine for `Fare` (5% missing) but distorted `Age` (21% missing): variance down a fifth, outliers from 7 to 69, because 148 filled ages all sit at one value and squeeze the middle half of the data.
- Always compute the fill value on the training set with `fit`, then `transform` both the training and the test set, because a fill value computed with the test rows is **data leakage** (G-535).
- `ColumnTransformer` lets each column use its own imputer (here the median for `Age` and the mean for `Fare`). Its `set_output` method can make the result a DataFrame, so the columns keep their names.
- So each gap in a numerical column gets one number from that column, from its centre to hide the gap or from its edge to mark it, and the before-and-after checks show whether the column survived.

8. Sources

Built from

- CampusX, "Handling missing data | Numerical Data | Simple Imputer", YouTube, https://www.youtube.com/watch?v=mCL2xLBDw8M
- Khan Academy, "Mean and standard deviation versus median and IQR", YouTube, https://www.youtube.com/watch?v=qNKOi08NxHs (the mean-against-median picture of section 2.1)

Other references

- scikit-learn User Guide, *Common pitfalls and recommended practices*, section "Data leakage".
- scikit-learn API reference, `SimpleImputer`.
- pandas API reference, `DataFrame.cov` and `DataFrame.corr`.

9. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Median imputation (G-1208)	Filling every gap with the median of the column's known values; better for skewed columns.
Arbitrary value imputation (G-208)	Filling every gap with one fixed value that never occurs, such as 99 or -1.
End of distribution imputation (G-686)	Filling every gap in a numerical column with a value at the far edge of its distribution, $\mu \pm 3\sigma$ for a normal column or $Q_3 + 1.5\,\mathrm{IQR}$ for a skewed one, so the model can tell which rows were missing.
Mean imputation (G-1197)	Filling every gap with the mean of the column's known values.
`fillna` (G-82)	The pandas method that replaces every `NaN` with a given value.
`statistics_` (G-145)	The fill values a fitted `SimpleImputer` has learned, one per column.
`fill_value` (G-81)	The value `SimpleImputer` uses with `strategy="constant"`.
`strategy` (SimpleImputer) (G-150)	The `SimpleImputer` parameter choosing the fill rule: mean, median, most_frequent or constant.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Target (G-1949)	The output column we predict, such as the class; also called the label.
Observation (G-1374)	One record of a dataset: one row of the data table.
Random sample imputation (G-1616)	Filling each gap with a value drawn at random from the column's known values.

Term	Meaning
Variance (of data) (G-2074)	The average squared distance of the values from their mean; the square of the standard deviation; divide by n for a population (NumPy default) or $n - 1$ for a sample (pandas default).
Complete case analysis (CCA) (G-425)	Dropping every row that has a missing value in any chosen column; also called listwise deletion.
MCAR (G-1192)	Missing completely at random: the gaps have no relation to any value in the data.
Data leakage (G-535)	Information from the test rows reaching the model during training, for example when a scaling or feature-selection step is fitted on all the data before the split; the test scores then come out higher than the model will really do on new data.
Covariance (G-496)	A number that says how two numerical features move together: positive if they rise together, negative if one rises as the other falls. Its size depends on the units.
Correlation (G-490)	A number from -1 to +1 that says how strongly two numerical features rise or fall together along a straight line: the covariance divided by both standard deviations, so the units drop out.
Missing indicator (G-1233)	A 0/1 (True/False) column recording whether a value was missing, added next to the imputed column so the model can learn whether being missing itself matters.
set_output (G-1780)	Method that, called as `set_output(transform="pandas")`, makes a scikit-learn transformer return a pandas DataFrame with column names instead of a bare NumPy array.